

Continuous Integration

Team 8: Ben and Bensons



Team Members: Alyssa Skipper, Benjamin Senior, Benson Chow, Chloe Ward, Florian Mengkris, Hannah Thomas, James Ingram, Olivia Spencer

Continuous Integration method and approaches

Our project began with the creation of a CI workflow and infrastructure, taking full advantage of different free services, to allow for seamless integration of work from our team members, and to maintain code quality, compatibility with different operating systems and software stability. Here are our methods and approaches:

- 1. Github organisation:**

Allow us to have centralised access control to the version control repositories. Ensuring security and easy access control changes, allowing for better productivity at no loss of security.

- 2. Github repositories:**

One of the popular Git hosts, it acts as our remote Git repository, allowing us to do branch based workflows, incremental commits and cloud based code backups. Branch based workflows allow for code reviews before pull requests, preventing bugs, enhancing code quality and making collaboration simpler. While incremental commits allowing for stability and traceability, allowing for changes that contain bugs to be easily reverted. Cloud-based code backup allowed us to not lose progress if individual members lost access to their local Git repository.

- 3. Github Actions:**

Github Actions allow us to run automatic workflows that are triggered on actions done to the version control repository. It allowed us to do automatic builds, testing and releases, generating reports and notifying members when there is a problem.

- 4. Gradle:**

Using the gradle wrapper, it allows us to have no dependency on local developer environments and same build setup across machines, even on machines with different operating systems or CPU architectures. It also handles the different build targets required for our libGDX game, such as the website and desktop JAR versions.

- 5. Discord:**

Allow us to assign tasks to different people for individual features or issues and allow them to comment on their progress. Our Discord is also integrated with our version control repository, for action tracking. It improves accountability, transparency and allows the team's members to stay aligned to the project goals and milestones.

In conclusion our CI approach and methods mostly adheres to industry standards but adjusted for our use case.

Discord is used instead of an industry-standard issue tracking system, due to our communication style, small team and time requirements.

Our reliance on Github and its mother organization Microsoft, is intentional, due to our project having limited confidentiality problems, eliminating the need to have complex self hosted solutions, and it is undoubtedly the most mature and widely adopted CI ecosystem, which we took full advantage of.

Gradle is used as our build automation system, inherited from the previous team, we did not change it, due to our time requirement, and it supported our existing project seamlessly.

Continuous Integration infrastructure

In depth technical report of our CI methods and approaches.

For Github Organisation and repositories:

- We created two Github repositories, for the website and game, having the same access controls, allowing members who are working on source code to have read and write access, while other members will only have read access.

For Github Actions and gradle:

- The Github action workflow is defined and split into two YAML files in the .github/workflow directory for maintainability and readability.
- Pseudo code for the logic in the YAML files:

```
For build-upload-release.yml:
Trigger conditions:
  On push to any branch or tags matching v[0-9]+.[0-9]+.[0-9]+
  On pull request to any branch
Job1 (Build JARs and upload if main/tag push, matrix on all OS):
  IF OS is Unix (ubuntu or macos):
    Make gradlew executable
  CHECK OS:
    Build JAR based on OS
    Upload jar 'lib-{OS}' from the correct path in OS FS
Job2 (create release on release tag push):
  IF tag starts with 'v'
    Download all the uploaded JAR files
    Create GitHub Release with files:
      - Game jar built from all 3 OS
```

```
For testing.yml
Trigger Conditions:
  On push
  On pull_request
  On workflow_dispatch
Job 1: Checkstyle Report
  Run Checkstyle on core module
  Upload checkstyle-report
Job 2: Testing and Test Coverage Report (matrix on all OS)
  IF OS is Unix (ubuntu or macos):
    Make gradlew executable
    Run headless tests on all OS
    Upload test reports 'test-reports-{OS}'
    Upload test result XML 'test-results-xml-{OS}'
  IF OS is ubuntu-latest:
    Build JaCoCo test report
    Upload JaCoCo coverage report
```

- Github actions used:
 - actions/checkout@v6 - used to checkout the repository,
 - actions/setup-java@v5 - used to setup java
 - actions/upload-artifact@v5 - used to upload artifacts
 - actions/download-artifact@v6 - used to download artifacts
 - softprops/action-gh-release@v2 - used to upload Github releases
- Gradle commands used:
 - ./gradlew html:dist - to build the GWT website
 - ./gradlew clean lwjgl3:build and ./gradlew.bat clean lwjgl3:build - to build JAR
 - ./gradlew core:checkstyleMain - to build the checkstyle report
 - ./gradlew test --continue and ./gradlew.bat test --continue - to run tests
 - ./gradlew jacocoTestReport - to build the jacoco test coverage report
- Utilizing Github workflows, we have followed an industry standard CI process , which can be easily expended on, for other testing frameworks and deployment methods.

For Discord:

- In our Discord server, we created a channel called Continuous Integration Logs where a HTTP webhook is exposed, by adding that webhook to our Github repository and with some configuration, Github will send raw JSON data on every single action done on the remote repository to the webhook, which is then forwarded to a formatting server before notifying us in a human readable format.